Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-68

Выполнил:

Студент 3-го курса

Группы АС-68

Дворак В. А

Проверил:

А. А. Крощенко
Ст. преп. кафедры ИИТ,

«___»_____ 2026 г.

Брест 2026

Вариант 4

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score

# 1. Загрузка данных
digits = load_digits()
X, y = digits.data, digits.target


# Архитектуры моделей
class MLPBase(nn.Module):
    def __init__(self, input_dim):
        super(MLPBase, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 10)
        )

    def forward(self, x):
        return self.model(x)


class MLPExp(nn.Module):
    def __init__(self, input_dim):
        super(MLPExp, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 32),
            nn.ReLU(),
            nn.Linear(32, 10)
        )

    def forward(self, x):
        return self.model(x)


# Функция для одного прогона (обучение + оценка)
def run_experiment(model_class, X_train_t, y_train_t, X_test_t, y_test_t,
epochs=100, lr=0.001):
    input_dim = X_train_t.shape[1]
    model = model_class(input_dim)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    final_loss = 0.0
    for epoch in range(epochs):
        model.train()
        y_pred = model(X_train_t)
        loss = criterion(y_pred, y_train_t)
```

```
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Сохраняется значение ошибки на последней эпохе
        if epoch == epochs - 1:
            final_loss = loss.item()

    model.eval()
    with torch.no_grad():
        y_logits = model(X_test_t)
        y_pred_class = torch.argmax(y_logits, dim=1)

    acc = accuracy_score(y_test_t.numpy(), y_pred_class.numpy())
    f1 = f1_score(y_test_t.numpy(), y_pred_class.numpy(), average='macro')

    return acc, f1, final_loss


# 2. Проведение 10 экспериментов с разным seed
N_RUNS = 10
base_accs, exp_accs = [], []
base_f1s, exp_f1s = [], []
base_losses, exp_losses = [], []

print(f"Запуск {N_RUNS} итераций эксперимента...\n")

for seed in range(N_RUNS):
    torch.manual_seed(seed)

    # Корректное разбиение
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=seed, stratify=y
    )

    # Корректная стандартизация без утечки данных
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Преобразование в тензоры
    X_tr_t = torch.tensor(X_train_scaled, dtype=torch.float32)
    X_te_t = torch.tensor(X_test_scaled, dtype=torch.float32)
    y_tr_t = torch.tensor(y_train, dtype=torch.long)
    y_te_t = torch.tensor(y_test, dtype=torch.long)

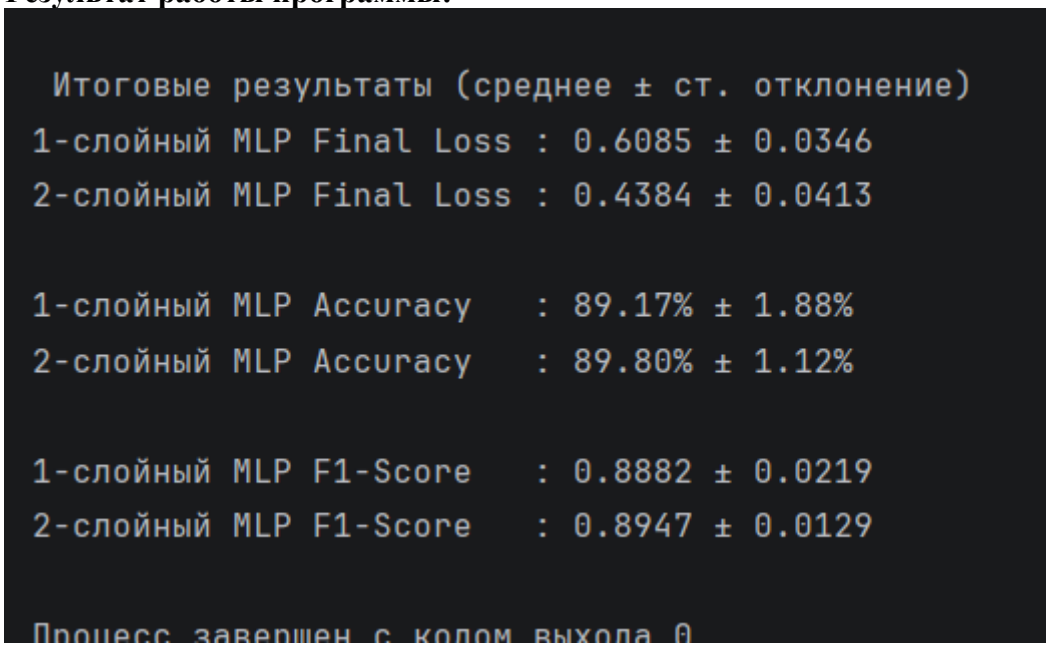
    # Обучение 1-слойной модели
    torch.manual_seed(seed)
    acc_b, f1_b, loss_b = run_experiment(MLPBase, X_tr_t, y_tr_t, X_te_t,
y_te_t)
    base_accs.append(acc_b)
    base_f1s.append(f1_b)
    base_losses.append(loss_b)

    # Обучение 2-слойной модели
    torch.manual_seed(seed)
    acc_e, f1_e, loss_e = run_experiment(MLPExp, X_tr_t, y_tr_t, X_te_t,
y_te_t)
    exp_accs.append(acc_e)
    exp_f1s.append(f1_e)
    exp_losses.append(loss_e)

# 3. Итоговая статистика по 10 прогонам
print(" Итоговые результаты (среднее ± ст. отклонение) ")
```

```
print(f"1-слойный MLP Final Loss : {np.mean(base_losses):.4f} ±
{np.std(base_losses):.4f}")
print(f"2-слойный MLP Final Loss : {np.mean(exp_losses):.4f} ±
{np.std(exp_losses):.4f}")
print("")
print(f"1-слойный MLP Accuracy   : {np.mean(base_accs) * 100:.2f}% ±
{np.std(base_accs) * 100:.2f}%")
print(f"2-слойный MLP Accuracy   : {np.mean(exp_accs) * 100:.2f}% ±
{np.std(exp_accs) * 100:.2f}%")
print("")
print(f"1-слойный MLP F1-Score   : {np.mean(base_f1s):.4f} ±
{np.std(base_f1s):.4f}")
print(f"2-слойный MLP F1-Score   : {np.mean(exp_f1s):.4f} ±
{np.std(exp_f1s):.4f}")
```

Результат работы программы:



Точнее оказалась 2-слойная модель (Accuracy 89.80% против 89.17%, F1-Score 0.8947 против 0.8882). Дополнительный скрытый слой позволил сети извлечь более сложные признаки цифр, что снизило ошибку (Final Loss 0.4384 против 0.6085) и уменьшило разброс результатов

Вывод: построил, обучил и оценил многослойный перцептрон (MLP) для решения задачи классификации